Django Complete Notes — Beginner to Advanced

Table of Contents

1. Django Fundamentals
2. Setup and Project Structure
3. URLs, Views, and Responses
4. Templates
5. Models and Database
6. ORM and QuerySets
7. Django Forms
8. Authentication and Authorization
9. Django REST Framework
10. Testing and Debugging
11. Security
12. Performance and Optimization
13. Reusable Django Apps and APIs
14. Deployment
15. Python Packaging
16. Git and Development Workflow
17. Interview Preparation
18. Practice Projects
19. Task Management REST API Capstone
20. Learning Roadmap

---

1. Django Fundamentals

What is Django?

Django is a high-level Python web framework used to build secure, maintainable web applications and APIs quickly.

Main Features

- Built-in authentication and authorization
- Object-Relational Mapper (ORM)
- Automatic admin interface
- URL routing
- Template engine
- Forms and validation
- Security protections
- Database migrations
- Testing tools
- Reusable applications

MVT Architecture

- **Model:** Handles data structure and database operations.
- **View:** Processes requests and returns responses.
- **Template:** Handles HTML presentation.
- **URL configuration:** Routes URLs to views.

Request flow:
`Browser → URL Resolver → Middleware → View → Model/Database → Response`

Django vs Flask vs FastAPI

Feature	Django	Flask	FastAPI
Type	Full-featured framework	Minimal framework	API-focused framework
ORM	Built in	External extension	External integration
Admin	Built in	Extensions	Extensions
Validation	Forms and model validation	Extensions	Type hints and Pydantic
Best use	Full web applications	Flexible small applications	Modern APIs and async services

Project vs App

- **Project:** The complete Django website and configuration.
- **App:** A module responsible for a particular feature.

Examples: `accounts`, `products`, `orders`, `tasks`.

WSGI vs ASGI

- **WSGI:** Traditional Python web-server interface.
- **ASGI:** Supports asynchronous applications and compatible long-lived connections.

Middleware

Middleware processes requests and responses globally.
Examples include:

- SecurityMiddleware
- SessionMiddleware
- AuthenticationMiddleware
- CommonMiddleware
- CsrfViewMiddleware

Middleware order matters. Request processing generally follows the configured order; response processing unwinds in reverse.

---

2. Setup and Project Structure

Install Django

```
python --version
python -m venv .venv
```

Activate the virtual environment:

```
# Windows PowerShell
.venv\Scripts\Activate.ps1

# Windows CMD
.venv\Scripts\activate.bat

# Linux/macOS
source .venv/bin/activate
```

Install packages:

```
python -m pip install --upgrade pip
python -m pip install Django djangorestframework
```

Create a Project and App

```
django-admin startproject config .
python manage.py startapp tasks
python manage.py runserver
```

Project Structure

```
project/
├── config/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── tasks/
│   ├── migrations/
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── manage.py
└── requirements.txt
```

Important Files

- `settings.py` : Project configuration.
- `urls.py` : URL routing.
- `views.py` : Request handling.
- `models.py` : Database models.
- `admin.py` : Admin configuration.
- `apps.py` : App configuration.
- `tests.py` : Automated tests.

- `manage.py` : Django command-line utility.
- `wsgi.py` : WSGI entry point.
- `asgi.py` : ASGI entry point.

Register an App

In `settings.py` :

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "rest_framework",
    "tasks",
]
```

Common Commands

```
python manage.py check
python manage.py runserver
python manage.py createsuperuser
python manage.py makemigrations
python manage.py migrate
python manage.py showmigrations
python manage.py test
python manage.py shell
python manage.py collectstatic
```

Environment Variables

Never commit production secrets to Git.

```
import os

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]

DEBUG = (
    os.environ.get("DJANGO_DEBUG", "False").lower() == "true"
)

ALLOWED_HOSTS = [
    host.strip()
    for host in os.environ.get("DJANGO_ALLOWED_HOSTS", "").split(",")
    if host.strip()
]
```

Use environment variables or an appropriately configured local environment file.

---

3. URLs, Views, and Responses

URL Routing

`config/urls.py` :

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("tasks/", include("tasks.urls")),
]
```

`tasks/urls.py`:

```
from django.urls import path
from . import views

app_name = "tasks"

urlpatterns = [
    path("", views.task_list, name="list"),
    path("<int:task_id>/", views.task_detail, name="detail"),
]
```

Common path converters:

- `str`
- `int`
- `slug`
- `uuid`
- `path`

Function-Based Views

```
from django.shortcuts import render, get_object_or_404
from .models import Task

def task_list(request):
    tasks = Task.objects.all()
    return render(request, "tasks/list.html", {
        "tasks": tasks
    })

def task_detail(request, task_id):
    task = get_object_or_404(Task, pk=task_id)
    return render(request, "tasks/detail.html", {
        "task": task
    })
```

Class-Based Views

```
from django.views.generic import ListView
from .models import Task

class TaskListView(ListView):
    model = Task
    template_name = "tasks/list.html"
    context_object_name = "tasks"
```

HttpResponse and JsonResponse

```
from django.http import HttpResponse, JsonResponse

def home(request):
    return HttpResponse("Hello Django")

def health(request):
    return JsonResponse({"status": "ok"})
```

Render and Redirect

```
from django.shortcuts import render, redirect

def home(request):
    return render(request, "home.html")

def redirect_home(request):
    return redirect("tasks:list")
```

Named URLs and Reverse

```
from django.urls import reverse

url = reverse("tasks:detail", kwargs={"task_id": 10})
```

Named URLs avoid hard-coded paths.

Query Parameters and Path Parameters

Example:
`/tasks/10/?status=done`

- Path parameter: `10`
- Query parameter: `status=done`

```
task_id = 10
status = request.GET.get("status")
```

Validate user input before using it.

Common HTTP Status Codes

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request

401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
409	Conflict
429	Too Many Requests
500	Internal Server Error

---

4. Templates

Template Variables

```
<h1>{{ task.title }}</h1>
<p>{{ task.created_at|date:"Y-m-d" }}</p>
```

Conditions and Loops

```
{% if tasks %}
    {% for task in tasks %}
        <p>{{ task.title }}</p>
    {% empty %}
        <p>No tasks available.</p>
    {% endfor %}
{% else %}
    <p>No tasks found.</p>
{% endif %}
```

Template Inheritance

`base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}My Site{% endblock %}</title>
</head>
<body>
    {% block content %}{% endblock %}
```

```
</body>
</html>
```

Child template:

```
{% extends "base.html" %}

{% block title %}Task List{% endblock %}

{% block content %}
<h1>My Tasks</h1>
{% endblock %}
```

Static Files

```
{% load static %}
<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

CSRF Protection

```
<form method="post">
    {% csrf_token %}
    <button type="submit">Submit</button>
</form>
```

Autoescaping

Django automatically escapes template variables by default.
Avoid using `|safe` with untrusted content. Unsafe HTML rendering can introduce Cross-Site Scripting (XSS).

---

5. Models and Database

Define a Model

```
from django.conf import settings
from django.db import models

class Task(models.Model):
    class Status(models.TextChoices):
        TODO = "todo", "To Do"
        IN_PROGRESS = "in_progress", "In Progress"
        DONE = "done", "Done"

    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.TODO,
    )
    owner = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="tasks",
```

```
    )
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [
            models.Index(fields=["owner", "status"]),
        ]

    def __str__(self):
        return self.title
```

Common Field Types

- `CharField`
- `TextField`
- `IntegerField`
- `BooleanField`
- `DecimalField`
- `DateField`
- `DateTimeField`
- `EmailField`
- `URLField`
- `UUIDField`
- `JSONField`
- `FileField`
- `ImageField`

null vs blank

- `null=True`: Database may store SQL NULL.
- `blank=True`: Validation may accept an empty value.

For optional text fields, `blank=True` without `null=True` is a common convention.

Relationships

```
class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )
    bio = models.TextField(blank=True)

class Category(models.Model):
    name = models.CharField(max_length=100, unique=True)

class Product(models.Model):
    name = models.CharField(max_length=100)
    categories = models.ManyToManyField(Category)
```

- `ForeignKey`: Many-to-one.
- `OneToOneField`: One-to-one.
- `ManyToManyField`: Many-to-many.

Migrations

```
python manage.py makemigrations
python manage.py migrate
python manage.py showmigrations
python manage.py sqlmigrate tasks 0001
```

- `makemigrations` : Creates migration files.
- `migrate` : Applies migrations.
- `showmigrations` : Shows migration status.
- `sqlmigrate` : Displays migration SQL.

Admin Registration

```
from django.contrib import admin
from .models import Task

@admin.register(Task)
class TaskAdmin(admin.ModelAdmin):
    list_display = ("id", "title", "owner", "status")
    list_filter = ("status",)
    search_fields = ("title", "description")
```

Create an administrator:

```
python manage.py createsuperuser
```

Constraints

Use database constraints for rules that must remain valid even when multiple requests or scripts write data concurrently.

```
class Coupon(models.Model):
    code = models.CharField(max_length=40, unique=True)
```

---

6. ORM and QuerySets

CRUD Operations

```
# Create
task = Task.objects.create(
    title="Learn Django",
    owner=user,
)

# Read
tasks = Task.objects.all()
task = Task.objects.get(pk=1)
tasks = Task.objects.filter(status="todo")
tasks = Task.objects.exclude(status="done")

# Update one object
task.status = "done"
task.save(update_fields=["status"])
```

```
# Update multiple rows
Task.objects.filter(status="todo").update(status="in_progress")

# Delete
Task.objects.filter(status="done").delete()
```

get() vs filter()

- `get()` returns one object.
- `get()` raises an exception if no object or multiple objects match.
- `filter()` returns a QuerySet, which may contain zero or more objects.

Useful Methods

```
Task.objects.exists()
Task.objects.count()
Task.objects.first()
Task.objects.last()
Task.objects.order_by("-created_at")
Task.objects.values("id", "title")
Task.objects.values_list("id", "title")
Task.objects.only("id", "title")
Task.objects.defer("description")
```

Field Lookups

```
Task.objects.filter(title__icontains="django")
Task.objects.filter(id__in=[1, 2, 3])
Task.objects.filter(due_date__isnull=True)
Task.objects.filter(title__startswith="Learn")
Task.objects.filter(id__gte=10)
```

Q Objects

Use `Q` for OR and complex conditions.

```
from django.db.models import Q

tasks = Task.objects.filter(
    Q(status="todo") | Q(status="in_progress")
)
```

F Expressions

Use `F` to perform database-side calculations.

```
from django.db.models import F

Inventory.objects.filter(pk=item_id).update(
    quantity=F("quantity") - 1
)
```

Add suitable conditions or database constraints to prevent invalid values.

Aggregation and Annotation

```
from django.db.models import Count
```

```
# Summary
Task.objects.aggregate(total=Count("id"))

# Per-status counts
Task.objects.values("status").annotate(
    total=Count("id")
)
```

- `aggregate()` : Produces summary values.
- `annotate()` : Adds calculated values to rows or groups.

Conditional Expressions

```
from django.db.models import Case, When, Value, IntegerField

tasks = Task.objects.annotate(
    rank=Case(
        When(status="in_progress", then=Value(1)),
        When(status="todo", then=Value(2)),
        default=Value(3),
        output_field=IntegerField(),
    )
).order_by("rank")
```

Avoid N+1 Queries

```
# ForeignKey and OneToOne relationships
Task.objects.select_related("owner")

# Many-to-many and reverse relationships
User.objects.prefetch_related("tasks")
```

- `select_related()` : Uses SQL joins for single-valued relations.
- `prefetch_related()` : Runs separate queries and combines results in Python.

Transactions

```
from django.db import transaction

with transaction.atomic():
    order = Order.objects.create(customer=customer)
    OrderItem.objects.create(
        order=order,
        product=product,
        quantity=1,
    )
```

Transactions protect related database operations. They do not automatically roll back external API calls or emails.

Raw SQL

Use parameterized SQL instead of string interpolation.

```
from django.db import connection

with connection.cursor() as cursor:
```

```
    cursor.execute(
        "SELECT id FROM tasks_task WHERE status = %s",
        ["todo"],
    )
    rows = cursor.fetchall()
```

Bulk Operations

```
Task.objects.bulk_create([
    Task(title="Task A", owner=user),
    Task(title="Task B", owner=user),
])

Task.objects.filter(owner=user).update(status="todo")
```

Bulk operations may bypass per-instance save behavior and signals. Review the relevant Django documentation.

Other ORM Topics

- `get_or_create()` and `update_or_create()`
- `Case` and `When`
- `Subquery`, `OuterRef`, and `Exists`
- Custom managers and QuerySets
- Database indexes
- Multiple databases and database routers
- Fixtures and data migrations
- Signals
- Abstract models and proxy models
- Multi-table inheritance
- Model validation and database constraints
- Database introspection
- Query optimization and SQL injection prevention

---

7. Django Forms

Form vs ModelForm

- `Form`: Manually defined fields and validation.
- `ModelForm`: Generates fields from a model and can save model instances.

```
from django import forms
from .models import Task

class TaskForm(forms.ModelForm):
    class Meta:
        model = Task
        fields = ["title", "description", "status"]

    def clean_title(self):
        title = self.cleaned_data["title"].strip()
        if len(title) < 3:
            raise forms.ValidationError(
```

```
                "Title must contain at least 3 characters."
            )
        return title
```

Validation

```
form = TaskForm(request.POST)

if form.is_valid():
    task = form.save()
else:
    errors = form.errors
```

Important methods and properties:

- `is_valid()`
- `cleaned_data`
- `errors`
- `clean_<field>()`
- `clean()`
- `save()`

commit=False

```
form = TaskForm(request.POST)

if form.is_valid():
    task = form.save(commit=False)
    task.owner = request.user
    task.save()
```

If the form includes many-to-many fields, call `form.save_m2m()` after saving the instance.

File Uploads

```
form = UploadForm(request.POST, request.FILES)
```

Use `enctype="multipart/form-data"` in HTML forms. Validate file size and content, and use safe storage practices.

---

8. Authentication and Authorization

Authentication vs Authorization

- Authentication identifies the user.
- Authorization determines what the user can do.

User Model

```
from django.contrib.auth import get_user_model

User = get_user_model()
```

Create users through the supported user manager:

```
user = User.objects.create_user(
    username="testuser",
```

```
    password="a-test-only-password",
)
```

Custom User Model

If a custom user model is needed, configure `AUTH_USER_MODEL` early in the project.

```
AUTH_USER_MODEL = "accounts.User"
```

Use `settings.AUTH_USER_MODEL` in model relationships and `get_user_model()` in application code.

Login Protection

```
from django.contrib.auth.decorators import login_required

@login_required
def dashboard(request):
    ...
```

Important Topics

- Password hashing
- Login and logout
- Password reset
- Sessions and cookies
- Groups and permissions
- Authentication backends
- Custom user models
- Object-level permissions
- JWT and token authentication
- Authorization checks

Never store plaintext passwords. Use Django's authentication framework.

JWT vs Session Authentication

- Session authentication is commonly suitable for browser-based applications.
- Token authentication is useful for API clients.
- JWT carries signed claims and may include expiry information.

JWT is not automatically more secure than sessions. Protect signing keys and token storage, and plan expiry, refresh, and revocation behavior.

---

9. Django REST Framework (DRF)

Installation

```
python -m pip install djangorestframework
```

Add `"rest_framework"` to `INSTALLED_APPS`.

API Function

```
from rest_framework.decorators import api_view
from rest_framework.response import Response

@api_view(["GET"])
```

```
def health(request):
    return Response({"status": "ok"})
```

Serializer

```
from rest_framework import serializers
from .models import Task

class TaskSerializer(serializers.ModelSerializer):
    class Meta:
        model = Task
        fields = [
            "id",
            "title",
            "description",
            "status",
            "created_at",
        ]
        read_only_fields = ["id", "created_at"]
```

Serializer Validation

```
def validate_title(self, value):
    value = value.strip()

    if len(value) < 3:
        raise serializers.ValidationError(
            "Title must contain at least 3 characters."
        )

    return value
```

Use `validate_<field>()` for field validation and `validate()` for cross-field rules.

CRUD API with Generic Views

```
from rest_framework import generics, permissions
from .models import Task
from .serializers import TaskSerializer

class TaskListCreateView(generics.ListCreateAPIView):
    serializer_class = TaskSerializer
    permission_classes = [permissions.IsAuthenticated]

    def get_queryset(self):
        return Task.objects.filter(owner=self.request.user)

    def perform_create(self, serializer):
        serializer.save(owner=self.request.user)


class TaskDetailView(generics.RetrieveUpdateDestroyAPIView):
    serializer_class = TaskSerializer
    permission_classes = [permissions.IsAuthenticated]
```

```
    def get_queryset(self):
        return Task.objects.filter(owner=self.request.user)
```

The queryset restrictions prevent users from accessing other users' tasks.

ViewSet

```
from rest_framework import viewsets, permissions
from .models import Task
from .serializers import TaskSerializer

class TaskViewSet(viewsets.ModelViewSet):
    serializer_class = TaskSerializer
    permission_classes = [permissions.IsAuthenticated]

    def get_queryset(self):
        return Task.objects.filter(owner=self.request.user)

    def perform_create(self, serializer):
        serializer.save(owner=self.request.user)
```

Router

```
from rest_framework.routers import DefaultRouter
from .views import TaskViewSet

router = DefaultRouter()
router.register("tasks", TaskViewSet, basename="task")

urlpatterns = router.urls
```

APIView vs ViewSet

- `APIView` : Explicit control over HTTP methods.
- `ViewSet` : Groups related actions and works with routers.
- `ModelViewSet` : Provides standard CRUD actions.

DRF Topics to Learn

- `Request` and `Response`
- `APIView` and `@api_view`
- Serializers and `ModelSerializer`
- Generic views and mixins
- ViewSets and routers
- CRUD endpoints
- HTTP status codes
- Pagination
- Filtering, searching, and ordering
- Authentication and permissions
- JWT and token authentication
- Throttling and rate limits
- API versioning
- File upload APIs
- Nested serializers

- Exception handling
- API testing
- OpenAPI and Swagger documentation

Pagination and Filtering

Configure pagination and filter backends deliberately. Whitelist supported filter and ordering fields, set a maximum page size, and document query parameters.

API Security

- Require authentication for protected resources.
- Restrict querysets to objects the current user may access.
- Assign protected fields such as `owner` on the server.
- Validate all incoming data.
- Apply appropriate throttling.
- Use HTTPS.
- Test cross-user access.

---

10. Testing and Debugging

Model Test

```
from django.test import TestCase
from django.contrib.auth import get_user_model
from .models import Task

User = get_user_model()

class TaskTests(TestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            username="testuser",
            password="test-only-password",
        )

    def test_task_string(self):
        task = Task.objects.create(
            title="Test Task",
            owner=self.user,
        )
        self.assertEqual(str(task), "Test Task")
```

Run Tests

```
python manage.py test
```

Testing Types

- Unit testing
- Integration testing
- API testing
- End-to-end testing
- Regression testing

DRF API Testing

DRF provides `APIClient`, `APITestCase`, and other testing helpers.

Test:

- Successful responses
- Invalid input
- Authentication
- Permissions
- Ownership
- Status codes
- Pagination and filtering
- Error handling

Debugging Best Practices

1. Read the full traceback.
2. Reproduce the error.
3. Check request data and settings.
4. Inspect database queries.
5. Write a regression test.
6. Log useful diagnostic information.
7. Never expose secrets or stack traces to API clients.

---

11. Security

Essential Security Practices

- Set `DEBUG=False` in production.
- Keep `SECRET_KEY` private.
- Configure `ALLOWED_HOSTS`.
- Enforce HTTPS.
- Use secure cookies.
- Protect session-authenticated forms with CSRF.
- Prevent SQL injection with ORM or parameterized SQL.
- Use template autoescaping to prevent XSS.
- Configure clickjacking protections.
- Enforce authentication and object-level permissions.
- Validate uploaded files.
- Keep dependencies updated.
- Restrict database and admin access.
- Use appropriate CORS settings.
- Back up production data.
- Configure logging and monitoring.

CSRF vs CORS

- **CSRF:** Protects against forged requests that exploit browser credentials.
- **CORS:** Controls whether browser JavaScript can read cross-origin responses.

They solve different problems. CORS is not authentication and does not replace CSRF protection.

Production Check

```
python manage.py check --deploy
```

Review the output using the actual production settings.

---

12. Performance and Optimization

Avoid N+1 Queries

```
tasks = Task.objects.select_related("owner")
```

Use `prefetch_related()` for many-to-many and reverse relations.

Database Indexes

Indexes can speed up queries but add storage and write overhead. Add them based on actual query patterns and query plans.

Caching

Common strategies:

- Per-view caching
- Template fragment caching
- Low-level cache API
- Reverse-proxy caching
- CDN caching

Never use a shared cache key for private user-specific data unless the key and policy safely account for user identity and permissions.

Other Optimization Topics

- Pagination
- Query profiling
- Database query plans
- Bulk database operations
- Static and media storage
- Background jobs
- Task queues such as Celery
- Connection and resource management

Measure performance before and after changing code.

---

13. Reusable Django Apps and APIs

Recommended Structure

```
tasks/
├── models.py
├── serializers.py
├── views.py
├── urls.py
├── permissions.py
├── services.py
├── tests/
└── migrations/
```

Use only the modules that improve clarity.

Best Practices

- Separate HTTP handling from core business logic.
- Keep validation explicit.
- Use reusable functions for genuinely shared behavior.
- Standardize error responses.
- Write API tests.
- Document configuration and dependencies.
- Version public APIs and packages deliberately.

Avoid adding unnecessary abstraction layers.

---

14. Deployment

Production Checklist

- `DEBUG=False`
- Environment-based secrets
- Correct `ALLOWED_HOSTS`
- HTTPS configured
- Production database configured
- Static files configured
- Media storage configured
- Migrations reviewed and applied
- Logging and monitoring enabled
- Backups tested
- Dependencies managed
- Tests pass
- Rollback plan documented

Static Files

```
python manage.py collectstatic
```

Configure `STATIC_ROOT` and an appropriate static-file server or storage backend.

WSGI and ASGI

- Gunicorn commonly serves Django WSGI applications.
- Uvicorn can serve Django ASGI applications.

Choose the server based on the application's deployment and async requirements.

Deployment Topics

- Production settings
- Environment variables
- Reverse proxies
- HTTPS and domains
- Database migrations
- Media storage
- Monitoring
- Backup and rollback strategies

Do not use Django's development server as the production server.

---

15. Python Packaging

Example Structure

```
my_package/
├── pyproject.toml
├── README.md
├── LICENSE
├── src/
│   └── my_package/
│       └── __init__.py
└── tests/
```

Build and Validate

```
python -m pip install --upgrade build twine
python -m build
python -m twine check dist/*
```

Important Topics

- `pyproject.toml`
- Package structure
- Build backends
- Distribution files
- Package metadata
- Local installation
- TestPyPI and PyPI
- Semantic versioning
- README and license
- Package testing

Test the built package in a clean environment before publishing. Never include credentials or private data in distributions.

---

16. Git and Development Workflow

Common Commands

```
git init
git clone <repository-url>
git status
git add .
git commit -m "Add task API"
git pull
git push
git switch -c feature/task-api
git merge feature/task-api
```

Example `.gitignore`

```
.venv/
__pycache__/
*.py[cod]
```

```
.env
.env.*
!.env.example
db.sqlite3
staticfiles/
.pytest_cache/
.coverage
htmlcov/
```

Adjust this to your project's needs.

Development Best Practices

- Use meaningful commit messages.
- Create focused branches.
- Review pull requests.
- Write tests.
- Follow PEP 8.
- Use a formatter and linter.
- Manage dependencies carefully.
- Document setup and deployment.
- Never commit secrets.

---

17. Interview Preparation

Q1. What is Django MVT?

Answer: Model represents data and database behavior, View processes requests and application logic, and Template renders the presentation.

Q2. What is the difference between a project and an app?

Answer: A project contains the overall configuration; an app is a module responsible for a specific feature.

Q3. What is middleware?

Answer: Middleware is a request-response processing layer for concerns such as sessions, security, and authentication.

Q4. What are migrations?

Answer: Migrations version and apply database schema changes. `makemigrations` creates files; `migrate` applies them.

Q5. What is QuerySet lazy evaluation?

Answer: Django often delays executing a QuerySet until results are needed. This enables query composition and can avoid unnecessary database operations.

Q6. `select_related()` vs `prefetch_related()`?

Answer: `select_related()` uses SQL joins for single-valued relationships. `prefetch_related()` uses separate queries and combines related results in Python.

Q7. `get()` vs `filter()`?

Answer: `get()` returns one object and raises exceptions for zero or multiple matches. `filter()` returns a QuerySet.

Q8. `annotate()` vs `aggregate()`?

Answer: `annotate()` adds calculated values to rows or groups. `aggregate()` produces summary values for the QuerySet.

Q9. Q objects vs F expressions?

Answer: `Q` builds complex boolean query conditions. `F` references database fields for database-side calculations.

Q10. ForeignKey vs OneToOneField vs ManyToManyField?

Answer: They represent many-to-one, one-to-one, and many-to-many relationships respectively.

Q11. What are signals?

Answer: Signals notify receivers when certain events occur. They are useful for some decoupled behavior but may hide side effects.

Q12. Authentication vs authorization?

Answer: Authentication identifies a user; authorization determines what that user may do.

Q13. FBV vs CBV?

Answer: FBVs are straightforward functions. CBVs provide reusable behavior through inheritance and mixins.

Q14. APIView vs ViewSet?

Answer: APIView gives explicit HTTP-method control. ViewSets group related actions and can generate routes through routers.

Q15. What is serializer validation?

Answer: It validates incoming API data before saving it. Use `is_valid()` and handle validation errors.

Q16. What does `transaction.atomic()` do?

Answer: It groups database operations into a transaction that can roll back if an exception occurs.

Q17. What is a database index?

Answer: An index can speed up certain queries but uses storage and increases the cost of writes.

Q18. How do you prevent users accessing other users' data?

Answer: Restrict querysets by the authenticated user, check object permissions, protect ownership fields, and test cross-user access.

Q19. How do you optimize a slow endpoint?

Answer: Measure query count and latency, inspect SQL, identify N+1 queries, review indexes, and optimize based on actual measurements.

Q20. How do you secure a Django API?

Answer: Enforce authentication and authorization, validate inputs, use HTTPS, protect secrets, configure CORS/CSRF appropriately, validate uploads, and monitor the application.

---

18. Practice Projects

Project 1: Blog Application

- CRUD posts
- Authentication
- Categories and tags
- Comments
- Pagination

- Search
- Tests
- Deployment

Project 2: Student Management System

- Students
- Courses
- Teachers
- Enrollments
- Database relationships
- Admin customization
- Search and filtering

Project 3: Task Management REST API

- Registration and login
- CRUD endpoints
- Ownership enforcement
- Status and priority
- Due dates
- Filtering and pagination
- API tests
- Documentation

Project 4: E-commerce API

- Products and categories
- Inventory
- Cart and orders
- Server-side price validation
- Transactions
- Payment integration
- Permissions
- Testing

Project 5: JWT-Protected User API

- Registration
- Login
- Token expiry
- Protected endpoints
- Permission tests
- Documentation

Project 6: Reusable Django Package

- Package structure
- `pyproject.toml`
- Unit tests
- README and license
- Versioning
- Build validation

- TestPyPI publishing

---

19. Task Management REST API Capstone

API Endpoints

Method	Endpoint	Purpose
POST	`/api/auth/register/`	Register user
POST	`/api/auth/login/`	Login
GET	`/api/tasks/`	List own tasks
POST	`/api/tasks/`	Create task
GET	`/api/tasks/<id>/`	Retrieve own task
PATCH	`/api/tasks/<id>/`	Update own task
DELETE	`/api/tasks/<id>/`	Delete own task

Authentication endpoints depend on the authentication implementation chosen.

Required Features

- User registration and login
- Task CRUD
- Status and priority
- Due dates
- Filtering and search
- Pagination
- Authentication
- Ownership permissions
- Serializer validation
- API documentation
- Environment-based configuration
- Deployment

Essential Security Rule

The client must not decide the task owner. Assign it on the server:

```
def perform_create(self, serializer):
    serializer.save(owner=self.request.user)
```

Restrict reads and writes:

```
def get_queryset(self):
    return Task.objects.filter(owner=self.request.user)
```

Testing Checklist

- Anonymous requests cannot access protected endpoints.
- Users can create their own tasks.
- Ownership is assigned by the server.
- Users cannot retrieve other users' tasks.
- Users cannot modify other users' tasks.
- Users cannot delete other users' tasks.
- Invalid input returns validation errors.
- Filters and pagination work.
- Protected fields cannot be mass-assigned.
- HTTP status codes are correct.
- Tests cover edge cases.

README Checklist

- Project overview
- Features
- Installation
- Environment variables
- Database setup
- Migration commands
- API endpoints
- Authentication instructions
- Running tests
- API documentation
- Deployment instructions

---

20. Learning Roadmap

Stage	Topics
1	Python fundamentals

2	HTTP, Git, and web basics
3	Django setup, URLs, views, and templates
4	Models, migrations, and ORM
5	Forms and authentication
6	Django REST Framework
7	Testing and security
8	Query optimization
9	Deployment
10	Portfolio projects and interview practice

Daily Study Routine

1. Review previous notes.
2. Learn one concept.
3. Write code yourself.

4. Solve a practical exercise.
5. Write a short explanation.
6. Commit progress to Git.
7. Review mistakes weekly.

Official Documentation

- Django: https://docs.djangoproject.com/
- Django Tutorial: https://docs.djangoproject.com/en/stable/intro/tutorial01/
- Django Security: https://docs.djangoproject.com/en/stable/topics/security/
- Deployment Checklist: https://docs.djangoproject.com/en/stable/howto/deployment/checklist/
- Django REST Framework: https://www.django-rest-framework.org/
- DRF Serializers: https://www.django-rest-framework.org/api-guide/serializers/
- DRF Permissions: https://www.django-rest-framework.org/api-guide/permissions/
- DRF Testing: https://www.django-rest-framework.org/api-guide/testing/
- Python Packaging: https://packaging.python.org/
- Git Documentation: https://git-scm.com/doc/

Final Advice

Do not only read the notes. Build projects, write automated tests, practise ORM queries, learn PostgreSQL, review API security, and deploy at least one complete application.
For backend job preparation, prioritize Django ORM, REST API development, authentication and authorization, testing, Git, PostgreSQL, and a well-documented portfolio project.